BRIGHTLOOP — Subscription Management System

Submitted by — Team Hackpack

Team Members: Krithika M, Akshayaa M, Amrutha Varshini P, Saimanga S

Date: Oct 31, 2026

Project Statement

The objective of this project is to replace spreadsheet tracking and manual billing with one Subscription Management System for a B2B SaaS business. Brightloop manages the full process in one place, from customer registration and quotations to subscription activation, recurring invoices, payment tracking, renewals and financial reporting. Admin configures the business, Sales, Operations and Finance staff perform their assigned work, and customers manage their own records in a separate portal. The Finance team can answer six month-end questions from a single dashboard: how many subscriptions are active, what revenue has been received, what is still owed, which invoices are overdue, which payments have arrived, and which invoices remain unpaid.

Modules Implemented

1. Project Setup and Requirements Understanding
2. Database Design
3. Authentication and Permissions
4. Admin Configuration
5. Quotations and Subscriptions
6. Billing and Payments
7. Dashboards and Reports
8. Backend API Development
9. Frontend Dashboard
10. Deployment and Documentation

---

1. Project Setup and Requirements Understanding

The purpose of this module was to understand the complete requirements and set up separate frontend and backend projects connected to PostgreSQL. This gave later modules a typed, runnable foundation.

Goals

- Replace spreadsheet tracking and manual billing with one integrated system.
- Connect registration, quotations, subscriptions, invoices, payments, renewals and reporting into one workflow.
- Give Admin, Internal Staff and Customers access only to what their role needs.
- Keep every page responsive within two seconds with thousands of subscriptions.

Key Performance Indicators (KPIs)

KPI	Description
Active Subscriptions	Subscriptions currently in Active status
Revenue Received	Sum of all completed payments
Outstanding Amount	Unpaid balance on confirmed invoices
Overdue Invoices	Confirmed invoices past their due date with a balance remaining
Subscriptions Expiring Soon	Active or paused subscriptions expiring within the configured window (default 30 days)
Page Response Target	Two seconds under normal operating conditions

---

Workflow

- Admin setup → Customer registration → Quotation creation → Customer approval → Subscription creation → Invoice generation → Payment recording → Renewal and closure → Financial reporting

---

Technology Stack

Layer	Technologies
Frontend	React, TypeScript, Vite, Tailwind CSS, React Router DOM, Axios, Recharts, Lucide React, React Hook Form, Zod
Backend	Node.js, Express.js, TypeScript, REST APIs, JWT, bcrypt, Nodemailer, PDFKit, Zod
Database	PostgreSQL through the Node.js pg package, using parameterized SQL and no ORM
Scheduling	In-process scheduler guarded by a PostgreSQL advisory lock (Redis and BullMQ were optional and not needed)

User Roles

Role	Profile	Responsibilities
Admin	Not applicable	Complete configuration, staff, catalogue, taxes, discounts, reports and settings
Internal Staff	Sales	Customers, quotations from templates, eligible discounts, quotation tracking
Internal Staff	Operations	Subscriptions, renewals, invoice operations, payment recording
Internal Staff	Finance	Revenue, outstanding balances, overdue invoices, payments and reports
Customer	Not applicable	Own plans, quotations, subscriptions, add-on requests, invoices, payments and account

---

2. Database Design

The PostgreSQL database uses 29 normalized tables and 41 indexes, with foreign keys and check constraints enforcing the business rules. The schema is in backend/sql/schema.sql and base configuration is in backend/sql/seed.sql.

Tables

Area	Tables
Identity	roles, staff_profiles, users, password_reset_tokens, customers
Catalogue	products, product_variants, subscription_plans, tax_rules
Discounts	discounts, discount_products, discount_plans, discount_usage
Quotations	quotation_templates, quotation_template_items, quotations, quotation_items, quotation_requests
Subscriptions	subscriptions, subscription_items, subscription_history, subscription_renewals
Billing	invoices, invoice_items, payments
System	notifications, audit_logs, settings, number_sequences

Data Integrity Rules

Rule	How it is enforced
Accurate money	NUMERIC(14, 2) columns, with calculations in integer paise
Unique accounts	Unique email constraint on users
Unique references	Unique constraints on quotation, subscription, invoice and payment numbers
Reference numbering	number_sequences rows, locked inside the transaction that uses them
One invoice per billing period	Partial unique index on subscription and period start, ignoring cancelled invoices
Balance consistency	outstanding and payment_status are generated columns on invoices
No overpayment	Check constraint that amount_paid cannot exceed total
Historical accuracy	Unit price, discount and tax details are stored on every document line
Timestamps	A trigger maintains updated_at on all main tables

Status Values

Record	Statuses
Quotation	draft, sent, accepted, rejected, expired
Subscription	draft, quotation, confirmed, active, paused, closed
Invoice	draft, confirmed, paid, cancelled
Invoice payment status	unpaid, partial, paid
Payment	completed, voided

---

3. Authentication and Permissions

Every protected request is authenticated with a JWT, then the user's role, permissions and account status are read from the database. A permission change or deactivation therefore applies immediately, and frontend role information is never trusted.

Authentication Rules

Rule	Implementation
Password policy	More than eight characters, with uppercase, lowercase and special characters
Password storage	bcrypt hashing with 12 rounds
Sessions	JWT lasting 8 hours, or 30 days with "Keep me logged in"
Logout	Revokes all of the user's tokens by raising a stored token version
Forgot password	Hashed, single-use reset token that expires after one hour
Account discovery	Forgot password responds identically whether or not the email exists
Rate limiting	30 attempts per IP address per 15 minutes on authentication endpoints
Google sign-in	Optional, and signs in existing accounts only

Default Permission Profiles

Permission	Sales	Operations	Finance
customers.view	Yes	Yes	Yes
customers.manage	Yes	No	No
products.view, plans.view	Yes	Yes	No
templates.view, discounts.view	Yes	No	No
quotations.view, quotations.manage	Yes	No	No
subscriptions.view	Yes	Yes	Yes
subscriptions.manage	No	Yes	No
invoices.view, payments.view	No	Yes	Yes
invoices.manage, payments.record	No	Yes	No
reports.view	No	No	Yes

Admin holds every permission and can change what each staff profile may do. Managing staff, taxes, discounts, the catalogue and settings always stays with Admin.

Customer Data Isolation

Portal routes accept no customer ID from the client and take it only from the verified session. A request for another customer's record returns 404, so it does not reveal whether the record exists. Draft quotations and draft invoices stay hidden from customers until they are sent or confirmed.

---

4. Admin Configuration

Admin defines the staff, catalogue, pricing rules and templates that every later workflow uses. Records are activated or deactivated instead of being deleted, so history stays intact.

Configuration Areas

Area	Key Fields	Functions
Staff	Name, email, phone, password, profile, status	Add, edit, activate or deactivate, send password reset
Customers	Name, company, email, phone, billing address, tax ID, status	Add, edit, activate or deactivate, view related records
Products	Name, type, sales price, cost price, description, add-on flag, status	Add, edit, activate or deactivate, manage variants
Subscription Plans	Price, billing frequency, minimum quantity, term length, availability dates, renewal rules	Add, edit, activate or deactivate
Quotation Templates	Default lines, validity days, terms and conditions, status	Add, edit, duplicate, preview, activate or deactivate
Taxes	Name, rate, target, description, status	Add, edit, activate or deactivate
Discounts	Type, value, minimum purchase and quantity, dates, usage limit, scope	Add, edit, activate or deactivate
Settings	Company, billing, numbering, notifications	Update company details, billing defaults, reference numbers and customer emails

---

Product Variant Pricing

The final product price is the sales price plus the variant's additional price. A variant marked as configurable takes its additional price per quotation, for negotiated tiers.

Variant (seed data)	Additional Price	Final Price
Basic	₹0	₹2,000
Professional	₹560	₹2,560
Enterprise	Configurable	Set per quotation

Plan Rules

Setting	Options	Effect
Billing frequency	Daily, weekly, monthly, yearly	Sets the invoice schedule
Term length	Number of billing cycles	Sets the expiry date
Renewal mode	Automatic, manual, none	Decides what happens at expiry
Automatic closure	On or off	Closes the subscription at the end of its period
Early closure and pausing	Allowed or not	Permits or refuses those actions

---

Pricing Calculation

The server looks up every catalogue price itself and does not accept unit prices from the client.

- **Line subtotal** = unit price × quantity
- **Taxable amount** = line subtotal − line discount
- **Line tax** = taxable amount × tax rate, for each active rule that applies to the line
- **Total** = subtotal − discounts + tax

A percentage discount applies to each eligible line. A fixed discount is capped at the eligible amount and shared across eligible lines by value, so no paise are lost to rounding.

---

5. Quotations and Subscriptions

An accepted quotation becomes a subscription carrying its exact prices, taxes and discount, and each quotation can be converted only once. Every status change is checked on the server and recorded in the subscription's history.

Quotation Workflow

1. A customer requests a quotation from the portal, or Sales starts one directly.
2. Sales selects a customer and a template, then adjusts the lines and an eligible discount.
3. The system shows live totals calculated on the server.
4. Sales saves a draft, which stays editable and hidden from the customer.
5. Sales sends it, which locks it and notifies the customer.
6. The customer accepts or rejects it, and can download its PDF.
7. Sales or Operations converts the accepted quotation into a confirmed subscription.

A quotation holds exactly one subscription plan and any number of products. A sent quotation past its expiry date becomes Expired and can no longer be accepted.

Subscription Lifecycle

- Draft → Quotation → Confirmed → Active → Closed
- Active ↔ Paused (only an Active subscription can pause; Resume returns it to Active)
- Draft, Quotation and Confirmed can also close directly; Closed is final
- A subscription created from a quotation starts at Confirmed; one created directly by staff starts at Draft

---

Subscription Actions

Action	Allowed from	Result
Move to Quotation	Draft	Quotation
Confirm	Quotation	Confirmed
Activate	Confirmed	Active, and the first due invoice is generated
Pause	Active	Paused, if the plan allows it
Resume	Paused	Active, with billing and expiry dates moved forward by the paused time
Close	Any status except Closed	Closed, final; closing an active or paused subscription early needs a plan that allows it
Renew	Active	Expiry extended by one term, if the plan allows renewal

Draft and quotation subscriptions can be edited. After confirmation the agreed details are locked.

Renewal and Closure at Expiry

Plan configuration	Behaviour at expiry
Automatic renewal	Extended by one term and recorded in the renewal history
Manual renewal	Stays active until staff renew it from the Renewals page
Automatic closure	Closed automatically
No renewal	Closed automatically

A late renewal continues from the old expiry date, so no billing period is skipped.

Add-on Requests

A customer's add-on request changes nothing until Operations approves it. An approved add-on joins the subscription from the next invoice, and invoices already issued do not change.

---

6. Billing and Payments

Invoices are generated automatically from each subscription's billing schedule as drafts, and staff confirm them before customers can see them. An invoice balance is always recalculated from its recorded payments, inside one database transaction.

Invoice Lifecycle

Draft → Confirmed → Paid, with Cancelled available from Draft or from Confirmed when no payments exist. Payment status (unpaid, partial or paid) is tracked separately from this lifecycle.

Action	Rule
Edit	Draft only; tax is recalculated at the rates captured when the invoice was generated
Confirm	Draft only; a zero-value invoice settles immediately
Send	Confirmed or Paid; emails the invoice with its PDF attached
Cancel	Refused if payments exist; releases any discount use the invoice took
Print or download PDF	Available to permitted staff, and to the customer once confirmed

---

Scheduled Billing Run

The scheduler runs when the API starts and every five minutes, and "Run billing now" on the Invoices page starts the same run. Each run performs these steps in order:

1. Expire sent quotations past their expiry date.
2. Apply expiry rules, renewing or closing subscriptions as configured.
3. Generate invoices for active subscriptions that are due, one per billing period.
4. Send renewal reminders for subscriptions expiring soon.

A subscription that is behind catches up with one invoice per missed period. Paused and closed subscriptions are not billed.

Payment Recording

1. Staff record the amount, method, date and notes against a confirmed invoice.
2. The system locks the invoice and refuses any amount above the outstanding balance.
3. It stores the payment with a unique reference.
4. It recalculates the amount paid and outstanding balance from all completed payments.
5. It marks the invoice Paid only when the full amount has been received.

Supported methods are bank transfer, cash, card and other. No payment gateway is integrated, as the requirements allow. Voiding a payment keeps it in the history and restores the invoice balance.

Discount Handling on Invoices

Eligibility is checked again for every invoice. An expired or used-up discount stops applying, and the subscription continues at full price. If a subscription has no valid discount of its own, the invoice takes the automatically applied discount that saves the customer the most.

Overdue Rule

An invoice is overdue when it is confirmed, its due date has passed and it still has an outstanding balance. A partial payment leaves it overdue until the balance is settled.

Risk Protection

Risk	Protection
Two invoices for one period	Row lock on the subscription, plus a unique database index
Overlapping billing runs	PostgreSQL advisory lock, including across server processes
Double-submitted payment	One-time idempotency key sent by each payment form
Converting a quotation twice	Unique link between a subscription and its quotation
Two invoices taking a discount's last use	Row lock on the discount while eligibility is checked

---

7. Dashboards and Reports

Each role has its own dashboard, and every value is calculated by SQL queries on current database records. Large summaries are aggregated in the database and never computed by loading records into the browser.

Role-Specific Dashboards

Dashboard	Summary Values	Charts and Tables	Filters
Admin	Customers, total, active and expiring subscriptions, revenue received, outstanding amount, overdue invoices, total invoices	Monthly revenue, subscription status, recent subscriptions, payments and invoices, overdue invoices, upcoming expirations	Date range, customer, subscription status, invoice status
Sales	Customers, total, pending, accepted, rejected and expiring quotations, customer requests	Recent customer activity	None
Operations	Total, active, expiring, pending activation and paused subscriptions, renewals due	Upcoming expirations, recent billing activity	None
Finance	Active subscriptions, revenue received, total payments, outstanding amount, overdue, paid and unpaid invoices	Invoiced and received by month, invoice status, overdue invoices, recent payments	Month, date range, customer, invoice status
Customer	Active subscriptions, available plans, pending quotations, upcoming expirations, unpaid invoices, total outstanding	Recent payments, upcoming expirations	None

---

Finance Questions Answered

Question	Where it is answered
How many subscriptions are active?	Active subscriptions card
How much revenue has been received?	Revenue received card and revenue report
How much money is still owed?	Outstanding amount card and outstanding report
Which invoices are overdue?	Overdue invoices table and overdue report
Which payments have been received?	Recent payments table and payment history report
Which invoices are still unpaid?	Unpaid invoices card and unpaid report

---

Reports

All 12 reports provide summary cards and a paginated table, and can be exported as CSV. The revenue report also includes a chart.

Report	Contents	Report-specific Filter
Total revenue	Invoiced, received and outstanding by month	None
Payment history	Every recorded payment	Payment status
All invoices	Every invoice	Invoice status, payment status
Outstanding payments	Confirmed invoices with a balance	Payment status
Overdue invoices	Past due with a balance	Payment status
Paid invoices	Settled in full	Payment status
Unpaid invoices	Confirmed and not yet settled	Payment status
Subscription summary	All subscriptions	Subscription status
Active subscriptions	Currently billing	None
Closed subscriptions	Ended subscriptions	None
Subscription renewals	Automatic and manual renewals	None
Customer summary	Subscriptions and balances per customer	None

Every report can be filtered by customer. All except the customer summary can also be filtered by month or date range.

Notifications and Audit Log

In-app notifications cover events such as quotations sent, invoices confirmed and payments received. The audit log records important administrative and financial actions, with who acted and when, and Admin can search it.

---

8. Backend API Development

The Express.js backend exposes 113 REST routes and keeps routes, controllers, services and SQL queries in separate layers. Business logic lives in services, and all database access uses parameterized queries through pg.

System Architecture

Every request passes through four backend layers to reach PostgreSQL:

- **React frontend** — Public website, internal application and customer portal; sends REST calls with a JWT
- **Routes and middleware** — JWT check, permission check, rate limit, error handling
- **Controllers** — Validate requests and send consistent responses
- **Services** — Business rules, pricing, lifecycle, billing, transactions
- **Repositories** — Parameterized SQL through the pg package
- **PostgreSQL** — 29 tables, with foreign keys, constraints and indexes
- **Billing scheduler** — On start, then every 5 minutes; uses the same services as the API

Nodemailer and PDFKit sit beside the services for customer emails and PDF files. The scheduler uses the same services as the API, so scheduled and manual billing follow identical rules.

Architecture

backend/
├── sql/
│ ├── schema.sql
│ ├── seed.sql
│ └── migrations/
├── src/
│ ├── server.ts
│ ├── config/ — database.ts, environment.ts
│ ├── routes/ — index.ts
│ ├── controllers/ — auth, admin, sales, billing, portal, report, system
│ ├── services/ — auth, admin, audit, pricing, quotation, subscription, billing, report, system
│ ├── repositories/ — user, customer, catalog, quotation, subscription, billing, report, system, lineItems
│ ├── middleware/ — auth, role, rateLimit, error
│ ├── validators/ — auth, catalog, common
│ ├── utils/ — calculation, dates, http, mailer, pdf, sql
│ ├── jobs/ — scheduler.ts
│ ├── scripts/ — dbSetup.ts, seedVolume.ts
│ └── types/ — index.ts
└── tests/ — calculation, workflow, performance

API Endpoints

All paths start with /api. Standard list, create, read, update and status routes are grouped below.

Method	Endpoint	Purpose
GET	/health, /public/plans, /public/info	Service health, available plans and company details, with no login
POST	/auth/register, /auth/login, /auth/google	Create an account or start a session
POST	/auth/forgot-password, /auth/reset-password	Request and use a password reset link
GET, PUT, POST	/auth/me, /auth/change-password, /auth/logout	Own profile, password and logout
GET, POST	/notifications, /notifications/:id/read, /notifications/read-all	Read notifications and mark them read
GET	/search, /dashboard/:kind	Global search and role dashboards
GET, POST, PUT, PATCH	/staff, /customers, /products, /plans, /taxes, /discounts, /templates	Manage configuration records and their status
GET, PUT	/staff-profiles	Read and edit staff permission profiles
POST	/staff/:id/reset-password	Send a staff password reset link
POST, PUT	/products/:id/variants	Add and edit product variants
POST	/templates/:id/duplicate	Copy a quotation template
GET, POST, PUT, DELETE	/quotations	Manage quotations; only drafts can be edited or deleted
POST	/quotations/preview	Calculate totals without saving
POST	/quotations/:id/send, /quotations/:id/convert	Send a draft and convert an accepted quotation
GET	/quotations/:id/pdf, /invoices/:id/pdf	Download a PDF
GET, POST	/requests, /requests/:id/decline, /requests/:id/approve-addon	Handle customer requests
GET, POST, PUT	/subscriptions	Manage subscriptions
POST	/subscriptions/:id/transition, /subscriptions/:id/renew	Change lifecycle status and renew
POST	/subscriptions/:id/generate-invoice	Generate the invoice that is due
GET, PUT	/invoices	List, read and edit draft invoices
POST	/invoices/:id/confirm, /invoices/:id/cancel, /invoices/:id/send	Invoice lifecycle actions
POST	/invoices/run-billing	Start the billing run immediately
GET, POST	/payments, /payments/:id/void	Record, list and void payments
GET	/reports/:type, /reports/:type/export	Run a report or export it as CSV

Method	Endpoint	Purpose
GET, PUT	/settings, /settings/:key, /settings/sequences/:code	Read and update settings and numbering
GET	/audit-logs	Search the audit log
GET, POST	/portal/*	Customer portal, always limited to the signed-in customer

---

Response Format

Result	Shape
Success	{ success: true, data }
Paginated list	{ success: true, data, meta: { total, page, pageSize, totalPages } }
Error	{ success: false, error: { code, message, details } }

Centralized error handling maps validation and database constraint errors to clear responses. Stack traces and passwords never reach the client.

Security Measures

- Parameterized queries throughout, with sort columns taken from a server-side whitelist.
- Zod validation on request bodies and query parameters.
- CORS limited to the configured client URL.
- Secrets and database credentials kept in environment variables.
- CSV exports neutralize spreadsheet formulas in customer-entered text.

Emails and PDFs

Nodemailer sends quotation, invoice, payment, renewal and password reset emails in the background, so a mail failure never fails the business operation. Without SMTP settings, emails are written to the server log. PDFKit generates quotation and invoice PDFs, which show "INR" because its built-in fonts have no rupee sign.

---

9. Frontend Dashboard

The React frontend provides a public website, an internal application and a separate customer portal, all connected to the backend APIs. Tables, forms, dialogs and filters are reusable components, and sidebars and routes follow each user's role.

Application Structure

frontend/src/
├── App.tsx
├── main.tsx
├── index.css
├── context/ — AuthContext.tsx, FeedbackContext.tsx
├── hooks/ — useAction.ts, useFetch.ts, useList.ts

```
├── layouts/  —  AppLayout.tsx
├── routes/  —  index.tsx
├── services/  —  api.ts
├── types/  —  index.ts
├── utils/  —  format.ts
├── components/
│   ├── charts/  —  index.tsx
│   ├── common/  —  ui.tsx, shared.tsx
│   ├── forms/  —  fields.tsx, LineEditor.tsx
│   └── tables/  —  DataTable.tsx
└── pages/
    ├── LandingPage.tsx
    ├── auth/  —  AuthPages.tsx
    ├── admin/  —  22 pages
    ├── staff/  —  Sales, Operations and Finance dashboards, Renewals, Profile
    └── customer/  —  PortalPages.tsx
```

Page Descriptions

Landing Page Public website with a split hero section, an illustrative finance preview using sample figures, features, workflow and FAQ. Its Pricing section lists the available plans from the database.

Authentication Pages Split-screen login, registration, forgot password and reset password pages, with "Keep me logged in", a password visibility toggle and an optional Google button.

Admin Dashboard Eight summary cards, monthly revenue chart, subscription status breakdown, recent activity tables, overdue invoices and upcoming expirations, with filters.

Users and Staff Staff accounts with add, edit, activate, deactivate and password reset actions, plus an editor for each profile's permissions.

Customers Searchable customer list and a detail page showing account information, balance summary and related quotations, subscriptions, invoices and payments.

Products and Subscription Plans Catalogue pages with search and filters, product variants, final price display, plan availability and renewal rules.

Quotation Templates Reusable default lines and terms. Admin manages them, while Sales can preview and use them.

Quotations List and customer requests tabs. The form has a customer search, template pre-fill, line editor, discount selection and live server-calculated totals.

Subscriptions List and form pages, plus a detail page with a lifecycle indicator, permitted actions, items, invoices, status history and renewals.

Renewals Subscriptions approaching expiry with one-click renewal, plus add-on requests awaiting approval.

Invoices List with status and payment filters, "Run billing now", and a detail page for editing, confirming, cancelling, sending, printing, downloading and recording payments.

Payments Searchable history with customer, method, status and date filters, plus recording and voiding.

Discounts and Taxes Admin pages for eligibility rules, usage limits, tax rates and tax targets.

Reports Report selector with month, date, customer and status filters, summary cards, paginated tables and CSV export.

Settings and Audit Log Company details, billing defaults, reference numbering, customer email settings, account settings and a searchable audit trail.

Customer Portal Dashboard, Browse Plans, My Quotations, My Subscriptions, Add-ons, My Invoices, Payments and Account Settings.

Navigation by Role

Role	Sidebar Pages
Admin	Dashboard, Customers, Products, Subscription Plans, Quotations, Quotation Templates, Subscriptions, Renewals, Invoices, Payments, Discounts, Taxes, Reports, Users and Staff, Settings, Audit Log
Sales	Dashboard, Customers, Products, Subscription Plans, Quotations, Quotation Templates, My Profile
Operations	Dashboard, Customers, Subscriptions, Renewals, Invoices, Payments, My Profile
Finance	Dashboard, Invoices, Payments, Revenue Reports, Outstanding Reports, Overdue Invoices, Subscription Summary, My Profile

The top bar shows global search for internal users, the theme toggle, notifications and a user menu with role information and logout.

State Management

- **AuthContext:** the signed-in user, login, logout and permission checks for showing or hiding UI.
- **FeedbackContext:** success and error notifications, and confirmation dialogs for important actions.
- **useList:** server-side search, filters, sorting and pagination, stored in the URL so a list can be linked to and survives a reload.
- **useFetch:** loading and error states, ignoring slower responses to older requests.

Design

The interface follows the supplied Brightloop designs, with an indigo gradient hero, Manrope type and an italic serif accent. The application itself uses a restrained light theme and Lucide icons, and the moon icon in the designs switches to an optional dark theme. Forms validate on both the frontend and the backend, using React Hook Form and Zod. Lists show loading, empty and error states.

---

10. Deployment and Documentation

The application runs locally against a PostgreSQL server, as the requirements specify, and has not been deployed to a hosting platform. All 33 automated tests pass, and all 40 measured API requests finished within the two-second target.

Deployment Architecture

Layer	Platform	Details
Frontend	Vite	React application at http://localhost:5173; npm run build creates the production files
Backend	Node.js and Express	REST API at http://localhost:5000/api, including the billing scheduler
Database	PostgreSQL 13 or newer	Locally installed server, database subscription_management
Version Control	Git and GitHub	Source code, schema, seed data and tests; .env files are excluded

---

Environment Configuration

backend/.env

Variable	Purpose
PORT	API port, default 5000
DATABASE_URL	PostgreSQL connection string
JWT_SECRET	Long random value that signs login tokens
CLIENT_URL	Frontend address, used for CORS and email links
EMAIL_HOST, EMAIL_PORT, EMAIL_USER, EMAIL_PASSWORD, EMAIL_FROM	SMTP settings; emails go to the server log if the host is empty
GOOGLE_CLIENT_ID	Optional, for Google sign-in
ENABLE_SCHEDULER	Set to false to stop the billing scheduler
AUTH_RATE_LIMIT	Optional override of the login attempt limit, used when running tests

frontend/.env

Variable	Purpose
VITE_API_URL	Backend API address
VITE_GOOGLE_CLIENT_ID	Optional, for Google sign-in

Setup Instructions

1. Install Node.js 20 or newer and PostgreSQL 13 or newer.
2. In backend, run npm install, copy .env.example to .env, and set DATABASE_URL and JWT_SECRET.
3. Run npm run db:setup to create the database, tables and base configuration.
4. Optionally run npm run db:seed:volume -- --customers 40 --subscriptions 120 for demo records.
5. Run npm run dev to start the API.
6. In frontend, run npm install, copy .env.example to .env, and run npm run dev.

The seed script creates one development account for each role, listed in the README. Change or remove them before any real deployment.

Testing

Test Area	Tests	Result	What It Covers
Calculations and PDF	9	Pass	Exact totals, discounts, tax targets, eligibility, billing dates, PDF layout
End-to-end API workflow	24	Pass	Quotation to payment, permissions, customer isolation, duplicate prevention, partial payments, overdue, renewal, closure, tax history
Total	33	Pass	Run on a clean database on 31 October 2026

The quotation-to-payment journey was also completed through the browser across the Sales, Customer, Operations and Finance roles. Each of those steps saved and displayed the correct totals.

Performance

Requests were timed one at a time on a development machine against a large synthetic dataset: 3,000 customers, 10,000 subscriptions, 84,771 invoices and 74,859 payments. These are API response times, not full page load times.

Request	Response Time
Admin dashboard (first load, slowest measured)	1,307 ms
Payment history report	412 ms
Payments list	350 ms
Revenue report	275 ms
Finance dashboard	200 ms
Invoices list	36 ms
Subscriptions list	12 ms

Known Limitations

- Logging out signs a user out on every device.
- The login rate limit is held in memory, per API process.
- Payment gateways are not integrated; staff record payments, as the requirements allow.
- Real email delivery and Google sign-in need the team's own credentials and have not been tested.

Remaining Checks

- Confirm the phone-width layout fixes on the dashboard and subscription pages.
- Review the dark theme and print layouts.
- Open a downloaded invoice PDF and review its appearance.

Deliverables

- Frontend source code in frontend/.
- Backend source code in backend/.
- PostgreSQL schema, seed data and migration folder in backend/sql/.
- Environment examples in backend/.env.example and frontend/.env.example.
- Automated tests and a performance script in backend/tests/.
- README with setup instructions, business rules and project layout.
- Project brief in project.md.
- This documentation, with architecture and lifecycle diagrams.
- GitHub repository link: to be added by the team after upload.